ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 17

Clases y objetos

- ¿Por qué objetos?
- El molde y la galleta: clase y objeto
- Métodos y $this
- El constructor
- Promoción de propiedades en el constructor
- Encapsulación: public y private
- Getters, setters y readonly
- Constantes de clase y static
- Una clase por archivo
- Los errores típicos de hoy



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Bienvenidos a la parte que da nombre al curso: la **programación orientada a objetos**, POO para los amigos. 🎉

Hasta ahora hemos programado de arriba abajo: datos en variables y arrays, y funciones que trabajan con ellos. Funciona, y lo seguiréis usando. Pero cuando un proyecto crece, los arrays sueltos y las funciones desperdigadas se vuelven difíciles de controlar. La POO propone algo muy sencillo de decir: **juntar los datos y lo que se puede hacer con ellos en una misma pieza**, el objeto.

No os voy a engañar: al principio cuesta un poco, porque es otra forma de pensar. Por eso hoy vamos despacio, con ejemplos pequeños, y cada punto añade **una sola idea** al anterior.

1 🤔 ¿Por qué objetos?

2 📘 El molde y la galleta: clase y objeto

3 👍 Métodos y $this

4 👌 El constructor

5 🚀 Promoción de propiedades en el constructor

6 ⚠️ Encapsulación: public y private

7 🔍 Getters, setters y readonly

8 😮 Constantes de clase y static

9 👌 Una clase por archivo

10 🐞 Los errores típicos de hoy

11 🚀 Ejercicios

12 👋 Lo que tenéis que llevaros de hoy



CLASE 17 · PUNTO 1

🤔 ¿Por qué objetos?

Mirad este código con un array, como los que hemos usado hasta ahora:

01-por-que-objetos.php — php-paso-a-paso/clase17 — Visual Studio Code

01-por-que-objetos.php

```php
<?php
$alumno = ["nombre" => "Marta", "nota" => 8.5];

$alumno["nota"] = 15;          // ¿un 15 sobre 10? PHP no protesta
$alumno["nomber"] = "Luis";    // una errata crea una clave nueva, sin avisar
```

PHP no protesta por ninguna de las dos cosas. Un array es una caja abierta: cualquiera puede meter cualquier cosa, con cualquier nombre. Y las funciones que trabajan con el alumno (`haAprobado()`, `calcularMedia()`...) andan sueltas por el código, lejos de los datos.

📄 Ejemplo: `01-por-que-objetos.php`

Un **objeto** soluciona las dos cosas a la vez [1]:

	Array	Objeto
Qué datos tiene	Los que le vayáis metiendo	Los que define su clase, ni uno más
Tipos	Cualquiera	Los que declara la clase (`float`, `string`...)
¿Quién puede cambiar los datos?	Cualquiera, en cualquier momento	Solo quien la clase permita, y comprobando que sean válidos
¿Dónde está lo que se puede hacer con él?	En funciones sueltas	En el propio objeto: `$alumno->haAprobado()`



CLASE 17 · PUNTO 2

📘 El molde y la galleta: clase y objeto

La analogía que mejor funciona en clase es la de la cocina:

- Una **clase** es el **molde** de galletas: dice qué forma tendrán todas.
- Un **objeto** es **cada galleta** que sale del molde. Todas tienen la misma forma, pero cada una es independiente: una puede llevar chocolate y otra no.

02-primera-clase.php — php-paso-a-paso/clase17 — Visual Studio Code

02-primera-clase.php

```php
<?php
declare(strict_types=1);

class Producto
{
    public string $nombre = "";
    public float $precio = 0;
}

$camiseta = new Producto();
$camiseta->nombre = "Camiseta";
$camiseta->precio = 15;

$gorra = new Producto();
$gorra->nombre = "Gorra";
$gorra->precio = 9.5;

echo "{$camiseta->nombre}: {$camiseta->precio} €";
```

📄 Ejemplo: `02-primera-clase.php`

Pieza	Qué es	En el ejemplo
`class Producto { }`	La **clase**: el molde. Su nombre empieza por **mayúscula**	`Producto`
`public string $nombre`	Una **propiedad**: un dato que tendrá cada objeto	`$nombre` , `$precio`
`new Producto()`	Crea un **objeto** (también se dice *instancia*)	`$camiseta` , `$gorra`
`$camiseta->nombre`	La **flecha** `->` entra en el objeto para leer o cambiar una propiedad	



💡 La flecha ya la habíais visto: en la clase 11 con `$fecha->format("d/m/Y")` y en la 15 con `$pdo->prepare(...)` . `DateTime` y `PDO` son clases que vienen con PHP. Hoy aprendéis a fabricar las vuestras.



CLASE 17 · PUNTO 3

👍 Métodos y $this

Un **método** es una función que vive **dentro** de la clase. Y dentro de un método, `$this` significa «este objeto»: el que ha recibido la orden [2].

03-metodos-this.php — php-paso-a-paso/clase17 — Visual Studio Code

03-metodos-this.php

```php
<?php
class Producto
{
    public string $nombre = "";
    public float $precio = 0;

    public function precioConIva(): float
    {
        return round($this->precio * 1.21, 2);
    }
}

echo $camiseta->precioConIva();   // 18.15 → usa el precio de la camiseta
echo $gorra->precioConIva();      // 11.5  → usa el precio de la gorra
```

Leed `$this->precio` como «**mi** precio». Cuando la camiseta ejecuta el método, `$this` es la camiseta; cuando lo ejecuta la gorra, `$this` es la gorra. El mismo código sirve para todas las galletas.

📄 Ejemplo: `03-metodos-this.php`



⚠️ **El olvido número uno:** escribir `$precio` en lugar de `$this->precio` dentro de un método. Recordad la clase 10: cada función tiene su habitación, y un método también. Para llegar a las propiedades del objeto hace falta `$this->`.



CLASE 17 · PUNTO 4

👌 El constructor

En el ejemplo anterior creábamos el objeto vacío y luego le íbamos dando los datos uno a uno. ¿Y si se nos olvida alguno? Para eso existe el **constructor**: un método especial, `__construct()`, que se ejecuta **solo** al hacer `new` y prepara el objeto [3].

04-constructor.php — php-paso-a-paso/clase17 — Visual Studio Code

04-constructor.php

```php
<?php
class Producto
{
    public string $nombre;
    public float $precio;

    public function __construct(string $nombre, float $precio)
    {
        $this->nombre = $nombre;
        $this->precio = $precio;
    }
}

$camiseta = new Producto("Camiseta", 15);
$gorra = new Producto("Gorra", 9.5);
```

Ahora el objeto **nace con sus datos**. Si alguien escribe `new Producto("Taza")`, PHP se para con un error: falta el precio. Mucho mejor que descubrirlo tres pantallas después.

📄 Ejemplo: `04-constructor.php`



💡 Los dos guiones bajos de `__construct` indican que es un **método mágico**: PHP lo llama solo, en un momento concreto. Hay más (`__toString`, `__get`…) y veremos alguno más adelante.



CLASE 17 · PUNTO 5

🚀 Promoción de propiedades en el constructor

¿Os habéis fijado en que en el ejemplo anterior escribimos cada dato **tres veces**? La propiedad arriba, el parámetro del constructor y la asignación `$this->nombre = $nombre`. Desde PHP 8 hay un atajo: declarar las propiedades **directamente en los parámetros** del constructor [4].

05-promocion-constructor.php — php-paso-a-paso/clase17 — Visual Studio Code

05-promocion-constructor.php

```php
<?php
class Producto
{
    public function __construct(
        public string $nombre,
        public float $precio,
        public int $stock = 0,
    ) {
    }
}

$camiseta = new Producto("Camiseta", 15, 20);
$gorra = new Producto("Gorra", 9.5);                    // stock: 0, el valor por defecto
$taza = new Producto(precio: 7.95, nombre: "Taza");     // argumentos con nombre
```

Hace exactamente lo mismo que el ejemplo 04, en la mitad de líneas. La clave está en la palabra `public` (o `private`) delante del parámetro: eso le dice a PHP «esto es también una propiedad; guárdala».

📄 Ejemplo: `05-promocion-constructor.php`



🎯 A partir de aquí usaremos siempre esta forma. Las llaves vacías `{ }` del final se quedan así: el constructor no tiene nada más que hacer. Fijaos también en la coma después del último parámetro: está permitida y facilita añadir parámetros más adelante.



CLASE 17 · PUNTO 6

⚠️ Encapsulación: public y private

Volvamos al problema del punto 1: cualquiera podía poner un 15 de nota. Con objetos lo evitamos **escondiendo** los datos delicados y ofreciendo **métodos** para manejarlos. Eso se llama **encapsulación** [5].

Visibilidad	Desde dónde se puede usar	En el restaurante
`public`	Desde cualquier sitio	La carta: la ve cualquier cliente
`private`	Solo desde **dentro** de la propia clase	La receta secreta del chef
`protected`	Desde la clase y sus clases «hijas»	Los secretos de familia (lo veremos con la herencia)

Una cuenta bancaria es el ejemplo perfecto: el saldo no puede cambiarse a mano, solo con ingresos y retiradas que comprueban las reglas.

06-encapsulacion.php — php-paso-a-paso/clase17 — Visual Studio Code

06-encapsulacion.php

```php
<?php
class CuentaBancaria
{
    public function __construct(
        public string $titular,
        private float $saldo = 0,
    ) {
    }

    public function ingresar(float $cantidad): void
    {
        if ($cantidad ⩽ 0) {
            throw new InvalidArgumentException("La cantidad debe ser positiva.");
        }
        $this->saldo += $cantidad;
    }

    public function retirar(float $cantidad): void
    {
        if ($cantidad > $this->saldo) {
            throw new RuntimeException("Saldo insuficiente.");
        }
        $this->saldo -= $cantidad;
    }

    public function getSaldo(): float
    {
        return $this->saldo;
    }
}

$cuenta = new CuentaBancaria("Ana García");
$cuenta->ingresar(100);
$cuenta->retirar(30);
echo $cuenta->getSaldo();       // 70

$cuenta->saldo = 1000000;       // ❌ Error: Cannot access private property
```

📄 Ejemplo: `06-encapsulacion.php`

¿Veis la ventaja? Es **imposible** que el saldo quede en negativo o que alguien se regale un millón: todas las puertas pasan por los métodos, y los métodos comprueban.



🎯 **Mi regla para empezar:** propiedades `private` por defecto. Solo las hacéis `public` si de verdad no pasa nada porque se cambien desde fuera.



CLASE 17 · PUNTO 7

🔍 Getters, setters y readonly

Cuando una propiedad es `private`, se suelen ofrecer dos tipos de métodos:

Tipo	Nombre habitual	Para qué
Getter	`getNota()`	Leer el valor
Setter	`setNota(float $nota)`	Cambiar el valor, **comprobando** que sea válido

07-readonly.php — php-paso-a-paso/clase17 — Visual Studio Code

07-readonly.php

```php
<?php
class Alumno
{
    public function __construct(
        public readonly int $id,
        public string $nombre,
        private float $nota = 0,
    ) {
    }

    public function getNota(): float
    {
        return $this->nota;
    }

    public function setNota(float $nota): void
    {
        if ($nota < 0 || $nota > 10) {
            throw new InvalidArgumentException("La nota debe estar entre 0 y 10.");
        }
        $this->nota = $nota;
    }

    public function haAprobado(): bool
    {
        return $this->nota >= 5;
    }
}
```

¿Y ese `readonly` del `id`? Significa «**solo lectura**»: la propiedad recibe su valor una vez, en el constructor, y después **nadie** puede cambiarla, ni desde fuera ni desde dentro [6]. Perfecto para cosas que no deben cambiar nunca, como el `id` de la base de datos.

07-readonly.php — php-paso-a-paso/clase17 — Visual Studio Code

07-readonly.php

```php
<?php
$marta = new Alumno(1, "Marta García");
echo $marta->id;       // 1 → se puede leer
$marta->id = 99;       // ❌ Error: Cannot modify readonly property Alumno::$id
```

📄 Ejemplo: `07-readonly.php`



💡 No hace falta un getter y un setter para cada propiedad. Pensad qué necesita el objeto. Una cuenta no tiene `setSaldo()`: tiene `ingresar()` y `retirar()`, que son lo que tiene sentido en el mundo real.



CLASE 17 · PUNTO 8

😮 Constantes de clase y static

Hay datos que no son de cada galleta, sino **del molde**: el IVA es el mismo para todos los productos. Para eso están las **constantes de clase** y los miembros `static` [7].

08-static-constantes.php — php-paso-a-paso/clase17 — Visual Studio Code

08-static-constantes.php

```php
<?php
class Producto
{
    public const IVA = 0.21;              // constante: del molde, y no cambia
    private static int $creados = 0;      // estática: compartida por TODOS los objetos

    public function __construct(
        public string $nombre,
        public float $precio,
    ) {
        self::$creados++;
    }

    public function precioConIva(): float
    {
        return round($this->precio * (1 + self::IVA), 2);
    }

    public static function totalCreados(): int
    {
        return self::$creados;
    }
}

echo Producto::IVA;              // 0.21 → sin crear ningún objeto
echo Producto::totalCreados();   // cuántos productos se han creado
```

Para llegar a...	Desde fuera	Desde dentro de la clase
Una propiedad o un método **de un objeto**	`$camiseta->precio`	`$this->precio`
Una constante o algo `static` (del molde)	`Producto::IVA`	`self::IVA`

Los `::` se llaman **operador de resolución de ámbito**; los alumnos lo llaman «los cuatro puntos». 😄

📄 Ejemplo: `08-static-constantes.php`



⚠️ No abuséis de `static`. Si todo es estático, en realidad estáis escribiendo funciones sueltas con otro nombre. Usadlo para lo que de verdad es del molde: constantes, contadores o métodos que **crean** objetos, como el `desdeArray()` del punto siguiente.



CLASE 17 · PUNTO 9

👌 Una clase por archivo

Hasta ahora hemos escrito la clase y el código que la usa en el mismo archivo, siempre con la clase **arriba del todo**. En un proyecto real, cada clase va en **su propio archivo**, que se llama **exactamente igual** que la clase:

Esquema

```
clase16/
├── clases/
│   └── Producto.php      ← class Producto { ... }
└── 09-una-clase-por-archivo.php
```

Y el archivo que la usa la carga al principio, como hacíamos con los archivos de funciones:

09-una-clase-por-archivo.php — php-paso-a-paso/clase17 — Visual Studio Code

09-una-clase-por-archivo.php

```php
<?php
declare(strict_types=1);

require_once __DIR__ . "/clases/Producto.php";

$filas = [
    ["nombre" => "Camiseta", "precio" => "15.00", "stock" => 20],
    ["nombre" => "Gorra", "precio" => "9.50", "stock" => 0],
];
$productos = [];

foreach ($filas as $fila) {
    $productos[] = Producto::desdeArray($fila);
}
```

Ese `desdeArray()` es un **método estático que fabrica objetos**: recibe una fila como las que devolvía PDO en la clase 15 y devuelve un `Producto`. Es el puente entre la base de datos y los objetos.

📄 Ejemplo: `09-una-clase-por-archivo.php` y la clase `clases/Producto.php`



💡 ¿Y cuando haya veinte clases? ¿Veinte `require_once`? No: en el bloque de Composer veréis la **autocarga**, que carga cada clase sola en cuanto la usáis. Por ahora, `require_once` al principio.



CLASE 17 · PUNTO 10

🐞 Los errores típicos de hoy

Mensaje (o síntoma)	Qué ha pasado	Solución
`Undefined variable $precio` dentro de un método	Falta `$this->`	`$this->precio`
`Cannot access private property`	Intentáis tocar algo `private` desde fuera	Usad el getter o el método que corresponda
`Cannot modify readonly property`	Intentáis cambiar una propiedad `readonly`	Si de verdad debe cambiar, no es `readonly`
`Too few arguments to function Producto::__construct()`	Falta un dato obligatorio al hacer `new`	Pasad todos los parámetros (o dadles un valor por defecto)
`must not be accessed before initialization`	Una propiedad con tipo que nunca recibió valor	Dadle valor en el constructor o un valor por defecto
`Class "Producto" not found`	Falta el `require_once`, o el nombre no coincide	Cargad la clase al principio y revisad mayúsculas
`Call to undefined method`	Método mal escrito, o `->` en lugar de `::` (o al revés)	`->` para objetos, `::` para `static` y constantes



CLASE 17 · PUNTO 11

🚀 Ejercicios

- ☐ **01 · Rectángulo.** Clase `Rectangulo` con base y altura (positivas: si no, excepción), y métodos `area()`, `perimetro()`, `esCuadrado()` y `describir()`. → `ej01-rectangulo.php`
- ☐ **02 · Cuenta con movimientos.** Mejorad `CuentaBancaria` para que cada ingreso y retirada guarde un movimiento (concepto, importe y saldo resultante) y mostrad el extracto en una tabla. Pista: un array `private` de movimientos y métodos `private` de ayuda. → `ej02-cuenta-movimientos.php`
- ☐ **03 · Conversor.** Una clase `Conversor` con métodos **estáticos** para pasar de grados Celsius a Fahrenheit y al revés, y de millas a kilómetros (con una constante de clase). → `ej03-conversor.php`
- ☐ **04 · Carrito.** Una clase `Carrito` que guarde objetos `Producto` con sus unidades, compruebe el stock, permita quitar productos y calcule el total con IVA. → `ej04-carrito.php` y `clases/Carrito.php`
- ☐ **05 · Alumnos.** Convertid unas filas de la tabla `alumnos` en objetos `Alumno` con `Alumno::desdeArray()`, mostrad la lista con una ✅ para los aprobados y calculad la media de los que tienen nota. → `ej05-alumnos.php` y `clases/Alumno.php`



🏆 **Reto:** en el proyecto CRUD de la clase 16, cread una clase `Curso` con `desdeArray()` y métodos como `precioConIva()` o `estaCompleto()`, y haced que el listado trabaje con objetos `Curso` en lugar de arrays.



CLASE 17 · PUNTO 12

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Clase y objeto	La clase es el molde; cada objeto, una galleta con sus propios datos
`new` y `→`	`new` crea el objeto; `→` entra en él
Método y `$this`	Una función dentro de la clase; `$this` es «este objeto»
Constructor	`__construct()` prepara el objeto al crearlo
Promoción	`public function __construct(private float $saldo)` declara y asigna a la vez
Encapsulación	Datos `private` y métodos públicos que comprueban las reglas
`readonly`	Se asigna una vez y ya no cambia
`const` y `static`	Del molde, no de cada galleta: `Producto::IVA`, `self::$creados`
Archivos	Una clase por archivo, con el mismo nombre, cargada al principio

📚 Fuentes y documentación oficial

1. Manual de PHP: clases y objetos
2. Manual de PHP: lo básico (class, new, $this)
3. Manual de PHP: constructores y destructores
4. PHP 8.0: promoción de propiedades en el constructor
5. Manual de PHP: visibilidad
6. Manual de PHP: propiedades (readonly)
7. Manual de PHP: la palabra clave static